

Ultimate Basic



Current version: **1.5.8**

A modern BASIC-like language that compiles directly to 6502 machine code for the **Commodore 64** and **Commodore 64 Ultimate**. It produces `.prg` files that run in VICE or on real hardware, and can also build `.d64` disk images.

Ultimate Basic looks like classic BASIC but compiles ahead of time — no interpreter, no line numbers required. It adds typed variables (`int` / `word` / `float` / `string` / arrays), subroutines and functions, structured control flow, and direct, high-level access to the C64's hardware: bitmap and block graphics, sprites, SID sound and music, raster/CIA/NMI interrupts, REU transfers, disk I/O, CRT cartridge export, and inline 6502 assembly.

Hires bitmap drawing (`plot`, `line`, `rect`, `circle`, `paint`) picks its color from `color pen c` — a persistent foreground color stamped into each touched cell. Multicolor bitmap mode has its own shape commands — `mplot`, `mline`, `mrect`, and `mcircle` — where the trailing `color` argument selects the 2-bit color source (0–3).

Sprite support includes hardware collision registers, side-effect-free software AABB tests with `box_hit()`, and frame selection from consecutive sprite animation data. Use `sprite_frame id`, `base`, `frame` inside an animation loop to select successive images stored in 64-byte slots; it changes the sprite image only and does not animate or move the sprite automatically.

Character tile maps can be embedded from self-describing `.ubmap` files, rendered as 40×25 viewports, queried and modified at runtime, in normal or multicolor text mode.

Koala Painter images can also be validated, embedded, shown in multicolor bitmap mode, and hidden again with `koala load`, `koala show`, and `koala hide`.

Build

```
cargo build --release      # binary: target/release/ub
cargo test                 # unit + integration tests
cargo install cargo-deb cargo-generate-rpm
cargo deb                  # Debian/Ubuntu package: target/debian/*.deb
cargo generate-rpm         # Fedora/RHEL package: target/generate-rpm/*.rpm
```

The Debian and RPM commands package the release binary as `/usr/bin/ub` and include the README and manual under `/usr/share/doc/ultimate-basic`. The same packages are also built automatically by the `linux-packages` GitHub Actions job on pushes to `main` and can be downloaded from its workflow artifacts.

Usage

```
ub build demo.ub -o demo.prg      # compile to a .prg (prints a memory map)
ub build demo.ub -v               # also print ZP layout + hex dump
ub build demo.ub --debug          # also write .sym, .dbg and .vs symbols
ub build demo.ub --asm            # also write a readable 6502 codegen listing
ub build demo.ub --d64 disk.d64   # also build a .d64 disk image
ub build demo.ub --d64 disk.d64 --add music.prg # embed extra files in the .d64
```

Option	Description
<code>-o, --output <file></code>	Output <code>.prg</code> (default: <code><input>.prg</code>)
<code>-v, --verbose</code>	Print full zero-page layout and a hex dump
<code>--no-stub</code>	Skip the BASIC <code>SYS</code> stub (code loads at <code>\$0801</code>)
<code>--debug</code>	Also produce KickAssembler <code>.sym</code> , C64Debugger <code>.dbg</code> , and VICE <code>.vs</code> files
<code>--asm</code>	Also produce a readable 6502 codegen listing as <code><output>.asm</code>
<code>--d64 [file]</code>	Also produce a <code>.d64</code> (default: <code><output>.d64</code>)
<code>--add <file></code>	Add an extra file to the <code>.d64</code> (repeatable)
<code>--explicit</code>	Require a <code>:type</code> annotation on every <code>var</code> , <code>sub</code> -param, and <code>fn</code> -param

With `--debug`, the compiler writes three files beside the program: an importable KickAssembler `.sym`, a C64Debugger/RetroDebugger `.dbg`, and a VICE monitor `.vs`. They contain the program boundaries, variables, arrays, subroutines, and BASIC labels. The `.dbg` export currently provides address symbols but not source-line stepping data.

With `--asm`, the compiler writes a readable assembly listing beside the program. Unlike a plain disassembly of the finished PRG, it is built from code-generation metadata and retains UB statement comments, zero-page variable names, array/subroutine/BASIC-label symbols, generated branch/JMP/JSR labels, instruction addresses, and emitted machine-code bytes. Compiler-generated helper routines are included in the same listing. The output uses KickAssembler syntax and can be assembled again. Known data regions—such as `data`, maps, sprite/character definitions, lookup tables, SID/Koala payloads, and `incbin` content—are emitted as `.byte` blocks instead of being mistaken for instructions.

What's new in 1.5.8

- **Initialised arrays:** `data <array>: v, ...` fill a declared `array` / `array_word` with constant values. Several lines continue where the previous one stopped, word arrays take 16-bit values (stored lo, hi), and the table is indexed like any array (`sinus[i]`). The values are stored after the code and copied into the array once at program start, right after the arrays are zeroed. These lines do not affect the `read` stream; more values than the array holds, or an undeclared name, is a compile-time error. This is what the Visual Assembler Curve Editor's "Export to UltimateBasic" generates.

```
var sinus = array(8)
data sinus: 128, 218, 255, 218
data sinus: 128, 37, 0, 37
sprite 0, x, sinus[i]
```

What's new in 1.5.7

New language elements:

- **charset on / charset off** switch the VIC-II to the RAM character set at `charset addr` (and back to the ROM set). Together with `chardef` this is a complete custom-font workflow. A bad base address, or program code that reaches into the set, is a compile-time error.
- **org addr** continues the subroutine code at a later address and leaves the gap free for data, so a big program can live around a charset, sprite data or a font.
- **incbin "file", addr, skip [, len]** skips a `.prg` load address (or any header) and can take just a slice of a file. `incbin ..., addr` may now target an `org` gap.
- **sfx channel, freq, frames [, wave]** is a non-blocking SID note: it returns at once and the envelope fades the note out, so it can be used inside a game loop (`sound` blocks).
- **tune ... end** describes a 3-voice SID tracker tune (instruments, order list and pattern rows such as "C-4 01 V24") directly in the source. The compiler builds a self-contained player at compile time - it uses no zero page - and defines `sid_init` / `sid_play`, so `music play|stop|pause|resume` work as with `load sid`. Vibrato, slide, note cut and speed

changes are supported. This is what the Visual Assembler SID editor's "Export to UltimateBasic" generates.

```
tune
  speed 6
  inst 0, $41, $09, $F0, $0800, $0064, $27, $1F
  order 0
  pat 0, 0, "C-4 00", "...", "E-4 00 V24", "...", "G-4 00", "... .. C00"
end
music play
```

```
charset $3800
incbin "font.bin", $3800      # data goes into the gap that `org` leaves
...
org $4000                    # (between two subs) code continues above the charset
charset on                   # the VIC-II now draws from $3800
sfx 1, $0500, 8, 128         # a short noise burst; the program keeps running
```

Fixes:

- **music play** stopped after the first tick: its CIA1 IRQ wrapper acknowledged the timer by writing `$01` to `$DC0D` (which clears the interrupt mask) instead of reading it.
- **Inline asm { }** dropped the addressing mode of label operands, so `LDA table,X`, `LDA table,Y` and `JMP (label)` were assembled as plain absolute instructions.
- **sound** lasted half as long as requested and crashed for durations other than 0 (a wrong branch patch). Now it waits on raster line 200 like `delay`.
- **word products** (`score += 300 * level`, `w = 100 * l + 900`) are now computed in 16 bits; they were truncated to 8 bits.
- **print of float expressions** (`print a / 10`) prints `N.DD` instead of the raw Q8.8 integer.
- **Arrays start zeroed** (RAM at power-up holds garbage).
- **Zero page overflow** is a compile-time error (`out of zero page`), and sequential `for` loops share their limit/step bytes instead of exhausting the permanent area.
- The manual no longer claims `%` binary literals (the lexer never supported them) and the `charset` example uses the right `$D018` value.

What's new in 1.5.6

- Added **color pen and multicolor shapes**. `color pen c` sets a persistent hires drawing color (0–15) that `plot`, `line`, `rect`, `circle`, and `paint` stamp into the foreground nibble of every cell they touch (the background nibble is preserved; default white). For multicolor bitmap mode, `mline`, `mrect`, and `mcircle` join `mplot`, each taking a trailing 2-bit `color` (0–3) source:

```

graphics on
gcls
color pen 2
circle 160, 100, 40    # red circle

graphics on multi
gcls
mcircle 80, 100, 40, 1  # multicolor circle
mline 0, 0, 159, 199, 3

```

The multicolor shapes reuse the same Bresenham/midpoint routines as their hires counterparts, plotting each pixel through `mplot`.

What's new in 1.5.5

- Added **Magic Desk CRT export**. Building with an output filename ending in `.crt` now writes a Commodore 64 cartridge image instead of a raw PRG:

```
ub build demo.ub -o demo.crt
```

The cartridge uses the same banked Magic Desk layout as VisualAssembler's CRT export: an 8×8K type-19 image with a boot loader in bank 0 and the compiled PRG payload copied into RAM on startup. Existing `.prg` and `.d64` output paths are unchanged.

- Added integration-test coverage for the CRT packer so the cartridge header, bank layout, payload copy, and boot stub stay stable.
- Added **`type ... endtype` structs**. Define a fixed layout of `int` (1 byte) or `word` (2 bytes) fields, then allocate an array of instances with the same syntax as any other array:

```

type TEntity
  var x: int
  var y: int
  var hp: word
endtype

const N = 8
var enemies: TEntity = array(N)  # N × 4 = 32 bytes at $C000

```

```

enemies[0].x = 5
enemies[i].hp = enemies[i].hp + 1

```

Constant indices fold to `LDA/STA absolute`; variable indices emit a shift-and-add `idx * elem_size` multiplier followed by `(ptr),Y` access. See `examples/type_demo.ub`. Struct-typed sub/fn parameters, default field values, and nested struct fields are not yet implemented.

- **Count-down `for / loop`** with negative constant `step` now works correctly. Previously the exit test was unsigned-only, so `for i = 20 to 0 step -2` either ran zero iterations or looped forever depending on whether it wrapped past 0. The compiler now picks the exit-branch encoding from the sign of the constant `step`, and emits an extra post-increment `BCS` check that catches the 8-bit underflow when counting down to (or through) 0.
- A `for / loop` with default `step +1` where `from > to` — e.g. `for i = 10 to 1` — used to skip the body silently. It is now a compile-time error directing you to `step -1`:

```

for-loop: from (10) > to (1) with default step +1 loops 0 times
  - use 'step -1' to count down

```

- Added the `--explicit` CLI flag. When passed to `ub build`, every `var`, `sub`-param, and `fn`-param declaration must carry a `:type` annotation; arrays and `const` are unaffected. The flag is a build-time switch, so nothing in the source changes to opt in or out. See `examples/explicit_demo.ub` and `examples/explicit_errors_demo.ub`.
- Added `examples/countdown_demo.ub` and `examples/countdown_errors_demo.ub`, plus new integration tests that both check the emitted branch encoding and run the loop on the test CPU to confirm it terminates.

What's new in 1.5.3

- Added **multi-dimensional arrays**. Declare an array with a comma-separated dimension list and index it the same way; storage is row-major and the total size is the product of the dimensions:

```

var grid = array(8, 8)      # 8x8 = 64 bytes
grid[r, c] = 42             # row-major: base + r*8 + c
var v = grid[r, c]
var wm = array_word(4, 4)  # word (16-bit) elements work too

```

Any number of dimensions is supported, dimensions may be `const` s, all-constant subscripts fold to a direct address at compile time, and a single subscript into a multi-dimensional array is still allowed as flat/linear access.

- Fixed PETSCII string encoding of `[`, `]`, and `^`, which previously printed as `?`. They now emit their correct C64 codes (`$5B`, `$5D`, `$5E`) in both the uppercase and lowercase charset modes.
- Added `examples/array2d_demo.ub` and integration tests for multi-dimensional arrays and the bracket encoding.

What's new in 1.5.2

- Added `ub build <file.ub> --asm` and `<file>.asm` codegen listings.
- Listings retain UB statement boundaries and compiler symbols instead of producing only a bare post-build disassembly.
- Zero-page variables and arrays are emitted as named assembler constants.
- BASIC labels, subroutines, relative branches, and generated in-program JMP/JSR targets receive readable labels.
- Every instruction includes its C64 address and generated machine-code bytes, making the output useful for inspection, debugging, optimization, and comparison with the PRG.
- Compiler helpers use descriptive `ub_helper_*` labels, while known embedded data is emitted with named `.byte` regions in reassemblable KickAssembler syntax.
- Added listing tests and documentation in the README, manual, CLI help, release notes, and compiler developer guide.

Documentation

The complete language and CLI reference lives in `**[MANUAL.md](MANUAL.md)**` – variables and types, operators, control flow, subroutines/functions, graphics (bitmap, double-buffered, block), sprites, sound and SID music, interrupts, REU, disk I/O, string/math functions, and inline assembly.

Release history is in `[whatnews.txt](whatnews.txt)`.

Examples

Ready-to-build demos are in `[`examples/`](examples/)` – bitmap and block graphics, sprite multiplexing, plasma and orbit effects, a flicker-free double-buffered 3D cube (``cube_demo.ub``), REU stash/fetch, SID music playback, scrollers, and more.